

Politecnico di Torino

Corso di laurea in Ingegneria Gestionale (L-8)

A.a. 2025/2026

Sessione di laurea di Novembre 2026

Ottimizzazione del piano di trasferte di una franchigia NBA con minimizzazione del costo economico

Relatore:
Prof. Giuseppe Bruno Averta

Candidato:
Francesco Guggia s325717

Sommario

0.Copia integrale della proposta di progetto 3

Studente proponente 3

Titolo della proposta 3

Descrizione del problema proposto 3

Descrizione della rilevanza gestionale del problema 3

Descrizione dei data-set per la valutazione 3

Descrizione preliminare degli algoritmi coinvolti 4

Descrizione preliminare delle funzionalità previste per l'applicazione software 5

1.Descrizione del problema affrontato 6

1.1.Semplificazioni adottate 6

2.Presentazione del data-set utilizzato 8

3.Strutture dati e algoritmi utilizzati 9

3.1.Il grafo delle arene 9

3.2.Le classi del modello 9

3.3.Ricostruzione dei viaggi reali 9

3.4.Modello di costo 9

3.5.Sequenziamento del singolo viaggio 10

3.6.Partizionamento delle trasferte in viaggi 10

3.7.Analisi fra le franchigie e centralità delle sedi 11

4.Diagrammi delle classi principali 12

5.Videate descrittive dell'applicazione 13

6.Tabelle e risultati sperimentali 14

6.1. Piano trasferte Portland Trail Blazers 15

6.2. Influenza del numero massimo di gare per viaggio 15

6.3. Confronto per centralità geografica 16

7.Valutazioni e considerazioni finali sui risultati ottenuti 17

7.1.Punti di forza 17

7.2.Punti deboli 17

7.3.Possibili sviluppi 17

7.4.Conclusioni 18

0. Copia integrale della proposta di progetto

Studente proponente

S325717 Guggia Francesco

Titolo della proposta

Ottimizzazione del piano di trasferta di una franchigia NBA con minimizzazione del costo economico.

Descrizione del problema proposto

L'idea da cui vorrei partire è una cosa che mi ha incuriosito, essendo io un giocatore di basket e appassionato di NBA: nel corso della stagione regolare ogni squadra disputa 41 partite in trasferta, distribuite su un territorio vasto dove l'organizzazione dei viaggi è fondamentale. L'insieme delle città da visitare e il numero di incontri contro ciascun avversario sono imposti dalla struttura del campionato e il mio obiettivo sarebbe ottimizzare l'insieme delle trasferte che una squadra deve affrontare.

Un conto molto grezzo: quattro trasferte a ovest (Denver, Phoenix, Los Angeles, San Francisco) affrontate come andate e ritorni separati da Philadelphia sono nell'ordine dei 27.500 km, mentre le stesse quattro raggruppate in un unico viaggio dovrebbero aggirarsi sugli 8.700 km.

Descrizione della rilevanza gestionale del problema

Il problema appartiene alla classe della pianificazione di risorse mobili con vincoli temporali, che è centrale in logistica.

Le leve che mi sembrano più interessanti sono due. La prima è di costo diretto: trasferimenti, pernottamenti e diarie sono una spesa proporzionale al numero di viaggi e alle distanze, ed è quella che scelgo di quantificare e minimizzare. La seconda è la disponibilità della risorsa umana: un piano più leggero: meno viaggi, meno chilometri dovrebbe alleggerire il carico di trasferta e la permanenza fuori sede, con un beneficio sulla condizione dei giocatori.

Descrizione dei data-set per la valutazione:

- Tabella team: le trenta franchigie della lega; identificativo, nome, città, stato, *conference* e divisione. È l'anagrafica fissa a cui si collegano partite, arene e distanze e con cui si popolano i menu dell'interfaccia.
- Tabella arena: gli impianti di gioco; identificativo, squadra proprietaria (*team_id*) e nome dell'arena. Una trentina di record; ogni arena è un nodo del grafo.

- Tabella distanza: la matrice delle distanze fra le sedi; la coppia di squadre (*team_a, team_b*) e i chilometri (km) che le separano, usati come pesi degli archi del grafo.
- Tabella stagione: le stagioni disputate; identificativo, anno di inizio e date di avvio e chiusura del campionato. Serve a filtrare partite e calendari per annata e a caricare più stagioni per verificare la stabilità dei risultati.
- Tabella partita: gli incontri di stagione regolare; identificativo, stagione di appartenenza (*stagione_id*), data, squadra di casa (*team_casa*) e squadra ospite (*team_ospite*) e il *flag sede_neutra* per le eventuali gare in campo neutro. È la tabella più popolosa (più o meno 1230 record per stagione), da dataset pubblici NBA.

I dati sono pubblici e non contengono informazioni personali. Non avendo trovato database scaricabili completi, intendo costruire il database in modo da avere le tabelle necessarie per il funzionamento dell'applicazione.

Descrizione preliminare degli algoritmi coinvolti

Le trenta arene sarebbero i nodi di un grafo non orientato, completo e pesato, con archi pesati sulla distanza. Una soluzione è una partizione ordinata delle trasferte: una lista di viaggi, ciascuno una sequenza di città che parte dalla sede della squadra e vi ritorna.

Il problema generale di costruire l'intero calendario di lega minimizzando i viaggi di tutte le squadre è il *Traveling Tournament Problem*, NP-difficile e fuori portata all'ottimo su trenta squadre. Avrei quindi scelto di isolarne il sottoproblema a singola franchigia.

Algoritmi che ho in mente:

1. *Sequenziamento del singolo viaggio*: una ricerca ricorsiva con *backtracking* e potatura sul costo parziale, per trovare l'ordine ottimo delle tappe. Limitando il numero di tappe per viaggio (direi 3-6, al più 7, per mantenere il risultato il più simile possibile a uno scenario reale, dato che nei calendari veri le trasferte non superano quasi mai le 6-7 gare).
2. *Partizionamento delle trasferte in viaggi*: è il punto che mi preoccupa di più, dato che non mi sono mai trovato davanti a un problema del genere durante il Corso. Mi sono informato e dovrei approcciare il problema in due fasi: la prima, un'euristica costruttiva che aggrega per vicinanza geografica, seguita da una ricerca locale che sposta una tappa da un viaggio a un altro finché riesce a migliorare, fermandosi a un ottimo locale.
3. *Funzione obiettivo e modello di costo*: traduco le metriche fisiche del piano (chilometri, tratte, notti, giorni fuori sede) in valore monetario applicando alcuni parametri di costo quota fissa e

quota per km del volo, costo del pernottamento, diaria giornaliera, dimensione della delegazione. Questi non sono un dato osservato ma valori configurabili e modificabili dall'interfaccia ed è la stima di costo che ne risulta la funzione che la ricerca locale cerca di minimizzare.

Semplificazioni che sto assumendo: restano fuori la disponibilità effettiva delle arene, i vincoli televisivi, le partite in sedi neutre e ogni componente non economica (fusi, affaticamento) non catturata dai costi. Il piano ottimizzato non assegna date, ma solo la sequenza delle gare. E, come detto, il partizionamento resta un'euristica (ottimo locale, non necessariamente globale), mentre solo il sequenziamento interno al viaggio è esatto.

Descrizione preliminare delle funzionalità previste per l'applicazione software

Vorrei realizzarla come applicazione desktop con interfaccia grafica in Python, libreria Flet, secondo il pattern MVC con un livello DAO per l'accesso al database, cioè lo schema che abbiamo usato a lezione.

L'idea di flusso è questa: l'utente sceglie squadra e stagione da menu popolati dal database; un comando analizza il calendario reale e ne mostra le metriche con l'elenco dei viaggi davvero compiuti; poi l'utente imposta il vincolo operativo, cioè il numero massimo di partite consecutive fuori sede per viaggio, e un secondo comando lancia l'ottimizzazione e restituisce il piano proposto, viaggio per viaggio.

Vorrei presentare il confronto come tabella comparativa fra calendario reale e ottimizzato, sia sulle metriche fisiche sia sul costo, così che il risparmio si legga in euro e non solo in chilometri. Lascerei che i parametri di costo siano modificabili dall'interfaccia. Mi piacerebbe aggiungere un confronto fra tutte le franchigie, che permetta all'utente di scegliere il parametro secondo cui fare il confronto, scegliendo tra: spesa se adottato il piano ottimizzato, spesa guardando il calendario reale e risparmio in caso una franchigia adottasse il piano calcolato dall'applicazione. Infine, inserirei la possibilità di stilare una classifica secondo la centralità geografica delle sedi, in modo da vedere che la posizione all'interno del territorio ha una grande influenza sulle trasferte e quindi sui costi.

1. Descrizione del problema affrontato

Ogni anno la NBA (*National Basketball Association*) si ritrova a dover organizzare il calendario stagionale delle partite sul vastissimo territorio di Stati Uniti e Canada, dove le singole trasferte possono raggiungere l'ordine di grandezza di migliaia di chilometri, comportando così una spesa veramente onerosa per le organizzazioni che ne fanno parte. Programmare una stagione significa anche fare in modo che il calendario sia economicamente efficiente e umanamente sostenibile. Ci sono, infatti, voci di spesa come il costo dei voli[1], degli hotel, dei pasti e del mantenimento generale di ogni singolo membro della squadra, giocatori e staff. Bisogna, però, considerare anche la componente umana, in quanto un calendario troppo denso e con trasferte lunghe molto ravvicinate pone dei seri rischi al recupero fisico e al mantenimento della salute mentale degli atleti.

L'obiettivo dell'algoritmo utilizzato è affrontare tale problema di logistica a risorse mobili con l'obiettivo di minimizzare il costo economico stimato, che dipende dai chilometri percorsi ma anche da tratte aeree, notti e giorni fuori sede. Questa problematica richiama il TTP (*Traveling Tournament Problem*), che nel caso della NBA, per come è strutturata l'organizzazione del calendario e i numerosi fattori da considerare, è fuori portata con metodi esatti.

La funzione dell'applicazione è quella di stilare il set di trasferte ottimo localmente per ogni singola squadra senza doverlo far combaciare con i set delle altre organizzazioni e senza assegnare alcuna data alle varie partite, in quanto non sono pubblicamente note le disponibilità delle arene e degli spazi televisivi riservati. Il risultato non è quindi un calendario giocabile, ma un riferimento teorico con cui confrontare il costo del calendario reale. Gli input sono squadra, stagione, numero massimo di gare per viaggio e parametri di costo; l'output è il piano di viaggi con le relative metriche e il costo, confrontabile con il calendario reale.

1.1 Semplificazioni adottate

Per rendere il problema più affrontabile sono state fatte le seguenti semplificazioni:

1. Il piano ottimizzato non assegna una data alle partite;
2. Le trasferte sono le stesse del calendario reale; il programma cambia solo come sono organizzate, non gli avversari o le città;
3. Ogni squadra viene analizzata da sola, senza tenere conto dei calendari delle altre squadre;
4. Viene presa in considerazione solo la stagione regolare;
5. Ogni viaggio parte dall'arena di casa e vi rientra. L'unico limite alla lunghezza di un viaggio è il numero massimo di partite scelto dall'utente;

[1] Presupponiamo che tutti gli spostamenti principali vengano fatti via aereo, in quanto nello scenario reale vengono affrontati per la maggior parte delle volte così e la vastità del territorio ne consente l'ipotesi.

6. Le partite in campo neutro sono state trattate come se fossero giocate nell'arena della squadra di casa;
7. Abbiamo supposto che tutti gli spostamenti vengano fatti in aereo e ho usato distanze fisse tra le arene;
8. Per calcolare i costi sono stati usati parametri fissi per voli, hotel e diaria, uguali in tutte le città e per tutta la stagione;
9. I giorni passati fuori casa sono stati calcolati nel seguente modo: un giorno per ogni partita più uno per ogni viaggio, sia per il piano sia per il calendario reale, in modo da poter confrontare i due risultati;
10. Per raggruppare le trasferte ho usato una ricerca locale, che cerca la soluzione migliore ma non garantisce di trovare quella migliore in assoluto. L'ordine delle tappe dentro ogni viaggio invece è quello ottimo.

2. Presentazione del data-set utilizzato

Il data-set utilizzato è il database "*nba_travel*", che abbiamo costruito con il supporto di *coding agents*, dato l'elevato numero di dati e l'assenza di un database scaricabile dal web, tramite uno *script* SQL. Il DB è formato da cinque tabelle: "team", "arena", "distanza", "stagione" e "partita", che in questo ordine contano 30, 30, 870, 5 e 6150 istanze dati per un totale di 7085 righe. Le partite provengono dalle pagine dei calendari di stagione regolare di Basketball-Reference (https://www.basketball-reference.com). Sono dati pubblici e non contengono informazioni personali.

La tabella "team" contiene le informazioni di ogni singola squadra che partecipa al campionato, identificate da un ID univoco. La tabella "arena" riporta le informazioni delle arene in cui vengono disputate le partite, nonché nodi del grafo di base utilizzato dall'applicazione, anch'esse identificate da ID univoco. La tabella "distanza" ci dà informazioni sulle varie distanze che separano ciascuna arena dalle altre. I dati sono stati calcolati tramite un ulteriore *script*, che utilizza la latitudine e la longitudine per trovare la distanza diretta tra due arene, in seguito abbiamo rimosso i dati che non servivano per il funzionamento dell'app e mantenuto solo la distanza, identificata dalla coppia di ID dei team che giocano nelle rispettive arene, dato che ogni team è associato a una singola arena, riconosciuta come quella di casa. La tabella "stagione" contiene informazioni sulle stagioni con le quali può essere fatto il confronto all'interno dell'applicazione. Ogni stagione è identificata da un ID univoco, al momento ne abbiamo caricate 5, ma virtualmente potrebbero essere inserite tutte, apportando delle modifiche per rispecchiare i cambiamenti subiti negli ultimi anni da arene, squadre e regolamento. Infine, la tabella "partita", la più popolata, contiene le informazioni di ogni singola partita disputata nella stagione e anch'essa è identificata da un ID univoco.

3. Strutture dati e algoritmi utilizzati

3.1 Il grafo delle arene

La struttura dati di base dell'applicazione è un grafo non orientato e pesato, costruito con la libreria *NetworkX*, in cui ogni nodo è un'arena e ogni arco collega due arene con un peso pari ai chilometri che le separano, presi dalla tabella "distanza". Dato che ogni arena è collegata a tutte le altre, il grafo è completo e presenta 30 nodi e 435 archi. Per passare velocemente dall'identificativo di una squadra al nodo corrispondente usiamo una *idMap*, cioè un dizionario che associa a ogni *team_id* la sua arena, così la distanza tra due sedi si ottiene leggendo direttamente il peso dell'arco che le unisce.

3.2 Le classi del modello

Tutte le classi del modello sono definite nel modulo *entita.py*. Oltre alle classi che rispecchiano le tabelle del database ("squadra", "arena", "stagione" e "partita") abbiamo definito quattro *dataclass* che descrivono il piano di trasferte. La classe "Viaggio" contiene le tappe in ordine di visita e i chilometri totali del giro sede – tappe – sede, più la data di inizio e di fine, che vengono valorizzate solo per i viaggi reali; da qui ricaviamo il numero di partite e il numero di tratte aeree, che sono sempre una in più delle partite, contando l'andata, gli spostamenti intermedi e il ritorno. "Metriche" raccoglie i valori fisici di un piano (chilometri, viaggi, tratte, trasferimenti, notti e giorni fuori sede), "Costo" le singole voci di spesa e "Piano" mette insieme squadra, stagione, lista dei viaggi, metriche e costo. Abbiamo usato la stessa struttura sia per il calendario reale sia per quello ottimizzato, in modo da poterli confrontare direttamente e mostrarli con le stesse tabelle.

3.3 Ricostruzione dei viaggi reali

Per il calendario reale leggiamo dal database tutte le partite della squadra nella stagione scelta, ordinate per data, e le scorriamo una alla volta: le gare in trasferta vengono aggiunte al viaggio in corso e la prima partita in casa lo chiude. I chilometri di ogni viaggio sono calcolati seguendo le tappe nell'ordine in cui sono state davvero giocate.

3.4 Modello di costo

La funzione obiettivo è il costo totale stimato della stagione di trasferte, che otteniamo partendo dalle metriche e applicando cinque parametri: quota fissa per ogni volo (25000 euro), costo per chilometro (30 euro), prezzo di una camera per notte (250 euro), diaria giornaliera per persona (150 euro) e

numero di persone della delegazione (25). Sono valori stimati e si possono modificare dall'interfaccia. Il costo totale è la somma di queste voci:

- Voli: tratte × quota fissa + chilometri × costo al km;
- Hotel: notti × prezzo della camera × persone della delegazione;
- Diarie: giorni fuori sede × diaria × persone della delegazione.

Dato che tratte, notti e giorni fuori sede crescono con il numero di viaggi, ogni viaggio in più porta con sé un costo fisso importante, per questo l'ottimizzazione tende a preferire meno viaggi ma più lunghi, anche a costo di qualche chilometro in più.

3.5 Sequenziamento del singolo viaggio

Fissate le tappe di un viaggio, l'ordine migliore per visitarle si trova con una *ricorsione* con *backtracking*: a ogni passo proviamo come tappa successiva ognuna di quelle non ancora visitate e, quando le abbiamo inserite tutte, aggiungiamo il ritorno a casa e confrontiamo i chilometri con il miglior percorso trovato fino a quel momento. Per non esplorare rami inutili usiamo una potatura sul costo parziale, cioè se i chilometri accumulati sono già maggiori o uguali a quelli del miglior percorso completo interrompo il ramo, dato che le distanze possono solo aumentare. Con al massimo 7 tappe gli ordini possibili sono 5.040 e con la potatura il calcolo resta molto veloce. In più salvo i risultati in una *cache*, indicizzata dalla sede e dall'insieme delle tappe, così se la ricerca locale mi chiede di nuovo lo stesso gruppo non lo dobbiamo ricalcolare. È l'unico passaggio dell'applicazione in cui la soluzione trovata è quella ottima.

3.6 Partizionamento delle trasferte in viaggi

Prendiamo le stesse trasferte del calendario reale e le dividiamo in gruppi di al massimo N partite, con N scelto dall'utente, lavorando in due fasi.

1. Euristica costruttiva: finché ci sono trasferte libere scegliamo come partenza di un nuovo viaggio quella più lontana dalla sede, poi aggiungiamo una alla volta la trasferta più vicina a una qualsiasi delle tappe già presenti nel gruppo, fino a riempirlo. Partire dalle città lontane serve a far viaggiare insieme le trasferte più costose, che sono quelle su cui si risparmia di più.
2. Ricerca locale**:** partiamo dai gruppi trovati e proviamo a spostare una tappa da un viaggio a un altro che non sia pieno e che non contenga già quella città, se un viaggio rimane vuoto lo eliminiamo. Ogni soluzione viene valutata con il costo totale, calcolando l'ordine ottimo di ogni viaggio con la ricorsione descritta sopra. Appena troviamo uno spostamento che abbassa

il costo lo accettiamo e ricominciamo da capo (*first improvement*), ci fermiamo solo quando nessuno spostamento riesce più a migliorare la soluzione, cioè in un ottimo locale.

Alla fine, ogni gruppo diventa un viaggio con le tappe nell'ordine ottimo.

3.7 Analisi fra le franchigie e centralità delle sedi

Nella sezione di analisi ripetiamo il calcolo del calendario reale e del piano ottimizzato per tutte le squadre e ordiniamo i risultati per costo reale, costo ottimizzato o risparmio. Per capire quanto la posizione della sede pesa sul costo usiamo invece il grado pesato di ogni nodo del grafo, cioè la somma delle distanze tra un'arena e tutte le altre: più è basso, più la sede è centrale rispetto al resto della lega.

4. Diagrammi delle classi principali

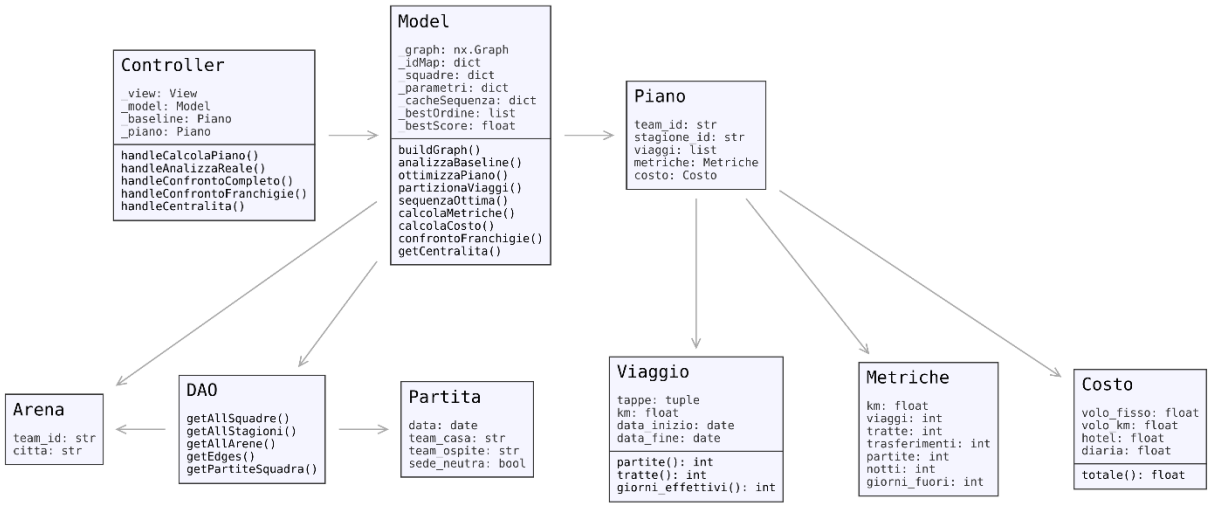


Figura 1: Diagramma delle classi.

L'applicazione segue il pattern MVC visto a lezione, con l'aggiunta del DAO per l'accesso al database. Nel diagramma le frecce indicano quale classe utilizza o crea un'altra. Il DAO si occupa del recupero dati dal database, il *model* contiene tutta la logica dell'applicazione, la *view* gestisce l'interfaccia grafica che l'utente visualizza e il *controller* fa da "passacarte" tra *model* e *view* occupandosi della comunicazione tra di essi.

5. Videate descrittive dell'applicazione

Di seguito vengono riportati degli screenshot delle schermate dell'applicazione, in modo da visualizzare ciò che è stato finora descritto.

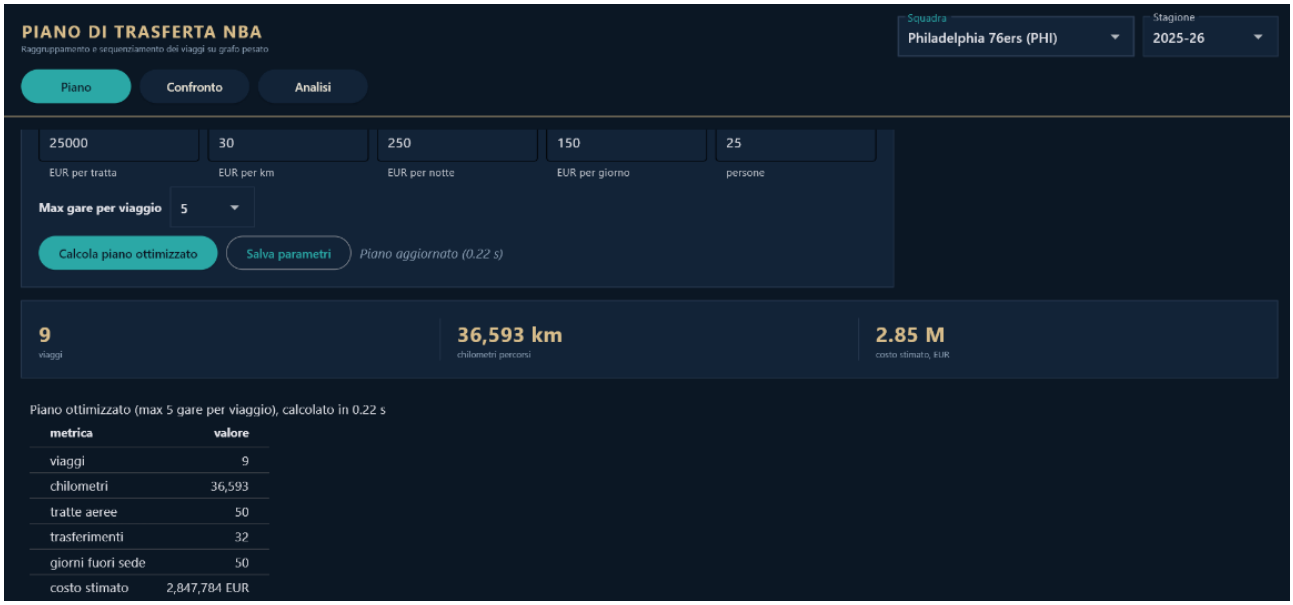


Figura 2: Schermata principale "Piano".

Una volta avviata l'applicazione (*figura 2*), l'utente viene portato nella scheda "Piano" dove deve selezionare squadra e stagione, inserire i parametri economici e definire il numero massimo di trasferte per viaggio, per poi premere il pulsante "calcola piano ottimizzato" per ottenere l'output con il set di trasferte e altri dati utili.

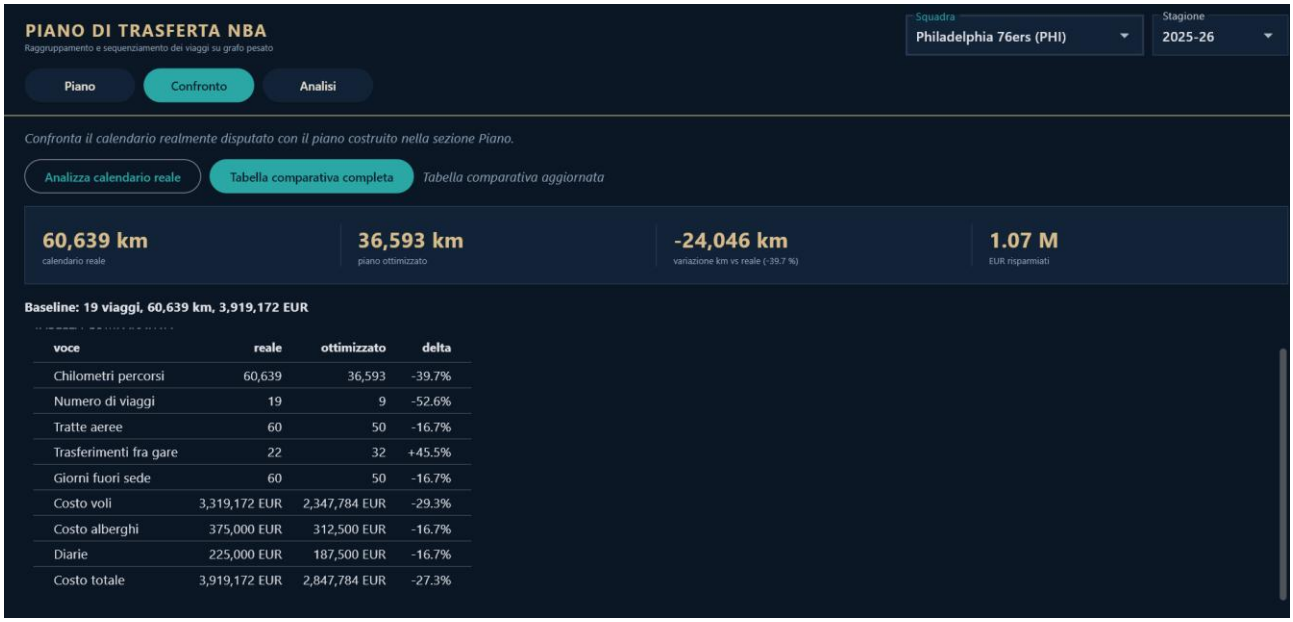


Figura 3: Schermata "Confronto".

In questa videata (*figura 3*) l'utente può eseguire un confronto con la stagione precedentemente selezionata, in modo da vedere le differenze e se c'è stato un miglioramento tra il piano reale e quello ottimizzato.

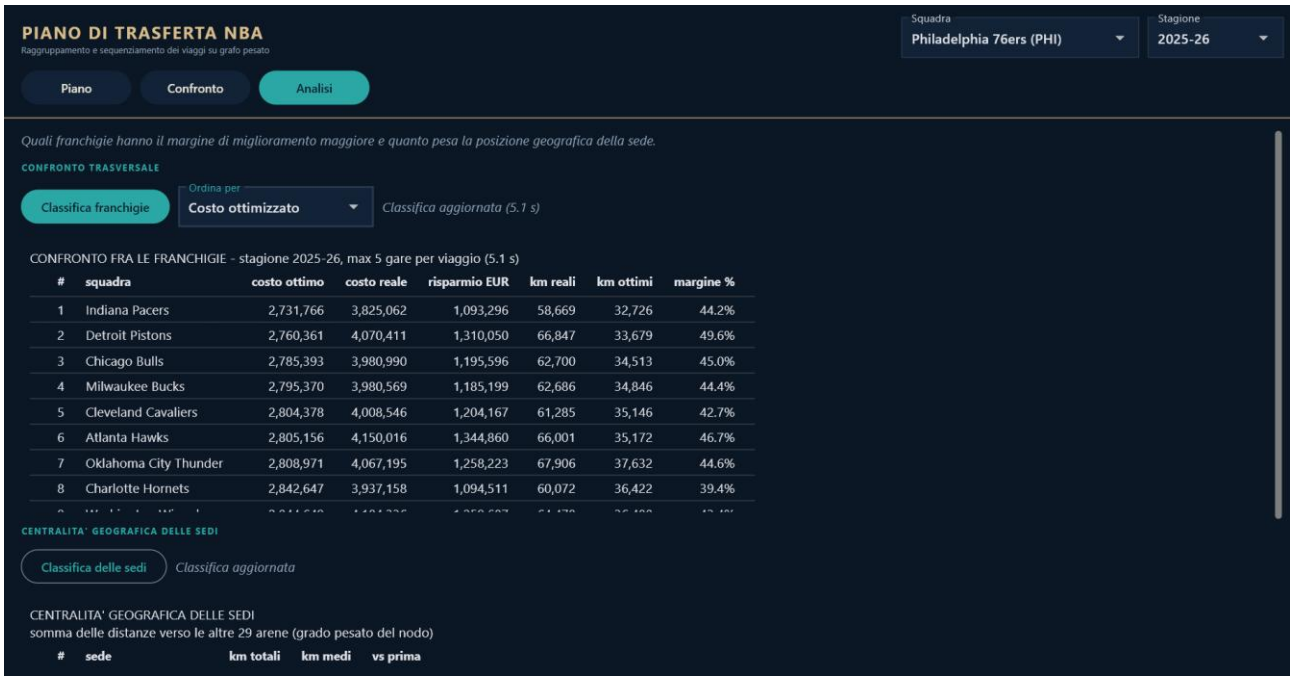


Figura 4: Schermata "Analisi".

In quest'ultima pagina (*figura 4*), è possibile classificare tutte le franchigie a seconda di diversi parametri, in modo da capire quali sono le organizzazioni che spendono di meno o risparmierebbero di più adottando il piano ottimizzato, in più viene mostrata una classifica stilata tenendo conto della centralità delle sedi, così da sottolineare che la posizione geografica ha un'enorme influenza sui costi da sostenere e chi parte da una posizione più favorevole è avvantaggiato.

Video dimostrativo del software: https://youtu.be/bqeyjp3MOpc

6. Tabelle e risultati sperimentali

6.1 Piano trasferte Portland Trail Blazers

Prendiamo in analisi il caso dei Portland Trail Blazers nella stagione 2025-26 e mettiamo a confronto i dati del piano reale e di quello ottimizzato utilizzando i parametri economici già di base inseriti senza modificarli e come numero massimo di gare per viaggio consideriamo 5:

Tabella 1. Confronto fra piano reale e piano ottimizzato nel caso dei Portland Trail Blazers.

Metrica	**Reale**	**Ottimizzato**	**Delta**
Km percorsi	82,237 km	58,950 km	-28.3%
Numero viaggi	15	9	-40%
Tratte aeree	56	50	-10.7%
Trasferimenti fra gare	26	32	+23.1%
Giorni fuori sede	56	50	-10.7%
Costo voli	3,867,107 EUR	3,018,509 EUR	-21.9%
Costo alberghi	350,000 EUR	312,500 EUR	-10.7%
Diarie	210,000 EUR	187,500 EUR	-10.7%
Costo totale	4,427,107 EUR	3,518,509 EUR	-20.5%

Possiamo notare che otteniamo un miglioramento in quasi tutte le voci, portando un risparmio sulla spesa totale di oltre 900.000 euro.

6.2 Influenza del numero massimo di gare per viaggio

Il numero massimo di gare per viaggio ha una grande influenza sui costi; infatti, se consideriamo ancora Portland e andiamo a mettere a confronto come variano i costi al variare del numero di gare otteniamo il seguente risultato:

Tabella 2. Confronto fra piano reale e piano ottimizzato al variare del numero massimo di gare per viaggio.

Numero gare per viaggio	**Reale (fisso)**	**Ottimizzato**	**Delta**
1	4,427,107 EUR	9,291,652 EUR	+109.9%
3	4,427,107 EUR	4,473,950 EUR	+1.1%
5	4,427,107 EUR	3,518,509 EUR	-20.5%
7	4,427,107 EUR	3,064,953 EUR	-30.8%

Vediamo che aumentando il numero di gare per viaggio l'organizzazione risparmia di più, però, considerando che un maggior numero di gare per viaggio potrebbe gravare di più sullo stato dell'atleta, non è sicuro che sia la scelta migliore.

6.3 Confronto per centralità geografica

Un altro confronto utile da riportare è quello tra i Portland Trail Blazers e gli Indiana Pacers, rispettivamente la squadra geograficamente più sfavorita e quella più avvantaggiata. Infatti, se calcoliamo il piano per entrambe le squadre nella stagione 2025-26 considerando gli assunti economici di base e un numero massimo di gare per viaggio pari a 5 e li mettiamo a confronto, otteniamo questi dati:

Tabella 3. Confronto fra la squadra geograficamente più avvantaggiata e quella più sfavorita.

	Km percorsi	**Costo**
Portland	58,950 km	3.52M
Indiana	32,726 km	2.73M
Differenza percentuale (Portland rispetto a Indiana)	+80.1%	+28.8%

È lampante che la squadra degli Indiana Pacers parta avvantaggiata sin dall'inizio quando vengono stilati i calendari delle trasferte.

7. Valutazioni e considerazioni finali sui risultati ottenuti

7.1 Punti di forza

I risultati mostrano che, lavorando solo su come le trasferte vengono raggruppate e ordinate, si possono ottenere risparmi teorici importanti. Nel caso di Portland, con al massimo 5 gare per viaggio, il costo totale scende del 20,5%, cioè oltre 900.000 euro in meno, e con 7 gare per viaggio si arriva a un risparmio del 30,8%.

Un altro aspetto positivo è la velocità: con un massimo di 5 gare per viaggio il piano di una singola squadra viene calcolato in meno di un secondo e anche la classifica di tutte le 30 franchigie richiede pochi secondi; quindi, l'applicazione si può usare per provare scenari diversi, cambiando i parametri economici o il numero massimo di gare per viaggio, e vedere subito come cambiano i risultati. Il confronto tra le franchigie e la classifica di centralità permette poi di guardare il problema non solo dal punto di vista della singola squadra ma della lega intera.

7.2. Punti deboli

Il limite più grande è che il piano ottimizzato non assegna le date alle partite, ovvero non costruisce un calendario, ma fornisce delle indicazioni su come organizzare gli itinerari. La ricerca locale poi si ferma a un ottimo locale, per cui non possiamo essere sicuri che non esista un raggruppamento migliore di quello trovato.

Anche i costi sono una stima: abbiamo usato gli stessi parametri per tutte le città e per tutta la stagione, mentre nella realtà un hotel a New York costa molto di più che a Memphis. Le partite in campo neutro, infine, sono trattate come se fossero giocate nell'arena della squadra di casa, perché il database non indica dove si sono disputate davvero.

Dalla tabella 2 si vede anche che con 1 o 3 gare per viaggio il piano ottimizzato costa più di quello reale. Questo succede perché il calendario vero contiene già alcuni viaggi più lunghi di 3 partite; quindi, con un vincolo così stretto l'applicazione è costretta a dividerli, aumentando i chilometri percorsi.

7.3 Possibili sviluppi

Il passo successivo sarebbe assegnare anche le date, rispettando i giorni di riposo, le partite in casa e la disponibilità delle arene, così da avvicinarsi al vero problema di costruzione del calendario. Si potrebbero poi usare costi diversi per ogni città, inserire nel database il luogo reale delle partite in campo neutro e aggiungere una penalità per le partite ravvicinate, in modo da tenere conto anche della stanchezza dei giocatori e non solo del costo economico.

7.4 Conclusioni

Con questo lavoro abbiamo voluto indagare quanto una pianificazione più attenta delle trasferte possa far risparmiare una franchigia NBA. I risultati mostrano che il margine di miglioramento c'è ed è consistente, ma anche che una parte del costo dipende dalla posizione della sede e non si può eliminare: anche con il piano migliore Portland spende circa il 29% in più di Indiana.

Il numero massimo di gare per viaggio resta una scelta da fare con attenzione, perché i viaggi lunghi fanno risparmiare ma pesano di più sui giocatori, e il giusto equilibrio tra le due cose dipende dalle priorità di ogni organizzazione.

Quest'opera è distribuita con licenza Creative Commons Attribuzione - Non commerciale - Condividi allo stesso modo 4.0 Internazionale (CC BY-NC-SA 4.0). Testo della licenza: https://creativecommons.org/licenses/by-nc-sa/4.0/deed.it